

WEEK 7 - PRACTICAL ASSESSMENT REPORT

WEB APPLICATION TESTING WITH BURP SUITE

VAPT PROJECT REPORT — PHASE 2

*Burp Suite Setup & Interception · Reconnaissance & Mapping · Manual Exploitation · Directory Discovery ·
Formal Vulnerability Reporting*

Prepared by	Md Mohtasim (DG/JULY/CYBER/017)
Role	Cyber Security Intern
Target Systems	http://localhost/dvwa/ · OWASP Juice Shop · testphp.vulnweb.com
Tools Used	Burp Suite Community Edition, Dirsearch / Gobuster, Firefox with FoxyProxy, Kali Linux
Covers Tasks	Task 22 – Task 26 (Week 7 Curriculum)
Submission Date	21 September 2026

1. Introduction & Objectives

This report is the second phase of the ongoing Vulnerability Assessment and Penetration Testing (VAPT) internship exercise. Week 6 covered the theoretical foundation of the OWASP Top 10 and an initial hands-on pass at DVWA using Burp Suite and OWASP ZAP. Week 7 builds directly on that foundation with a structured, tool-driven workflow centred entirely on Burp Suite Community Edition: proxy interception, application mapping, manual exploitation with Repeater and Intruder, endpoint/directory discovery, and formal vulnerability write-ups suitable for a professional VAPT report.

The objective of this phase is to demonstrate practical proficiency in:

- Configuring Burp Suite as an intercepting proxy and trusting its CA certificate for HTTPS traffic.
- Mapping an application's attack surface (pages, forms, parameters) using the Target and Proxy tabs.
- Manually testing and confirming SQL Injection, XSS, weak authentication, and parameter tampering using Repeater and Intruder.
- Performing directory and hidden-endpoint discovery with Burp and dedicated tools (Dirsearch / Gobuster).
- Documenting at least three confirmed vulnerabilities in a real-world VAPT report format — name, description, reproduction steps, proof, impact, and remediation.

Scope & Authorisation

All testing in this exercise was performed exclusively against applications that are intentionally vulnerable and explicitly provided for security training and legal testing:

- DVWA (Damn Vulnerable Web Application) — running locally at <http://localhost/dvwa/>.
- OWASP Juice Shop — a modern intentionally-vulnerable web application, run locally or via the public demo instance.
- testphp.vulnweb.com — Acunetix's publicly available, legally testable demo target.

No production system, third-party application, or system without explicit authorisation was accessed or tested at any point during this exercise.

2. Task 22 — Burp Suite Setup & Interception

2.1 Installation

Burp Suite Community Edition was downloaded from the official PortSwigger website (portswigger.net/burp/communitydownload) and installed on the testing machine. Community Edition provides the Proxy, Repeater, Intruder (rate-limited), Target, and Decoder modules used throughout this exercise.

2.2 Browser Proxy Configuration

Firefox was configured to route all traffic through Burp's listener using the FoxyProxy extension, which allows the proxy to be toggled on/off per tab without changing the OS-wide network settings.

Setting	Value	Purpose
Proxy listener	127.0.0.1 : 8080	Burp's default intercepting proxy address
Browser proxy (FoxyProxy)	127.0.0.1 : 8080 (HTTP & HTTPS)	Routes all browser traffic through Burp
Intercept toggle	Proxy → Intercept → Intercept is on	Pauses each request for manual review

2.3 Installing the Burp CA Certificate (HTTPS Interception)

To intercept HTTPS traffic without browser certificate warnings, Burp's self-signed CA certificate was installed into Firefox's trust store:

- With the browser proxy pointed at Burp, <http://burp> was visited and the CA Certificate was downloaded (cacert.der).
- In Firefox: Settings → Privacy & Security → Certificates → View Certificates → Authorities → Import, and the downloaded certificate was trusted for "Identify websites".
- HTTPS sites subsequently loaded through the proxy without certificate errors, confirming successful interception.

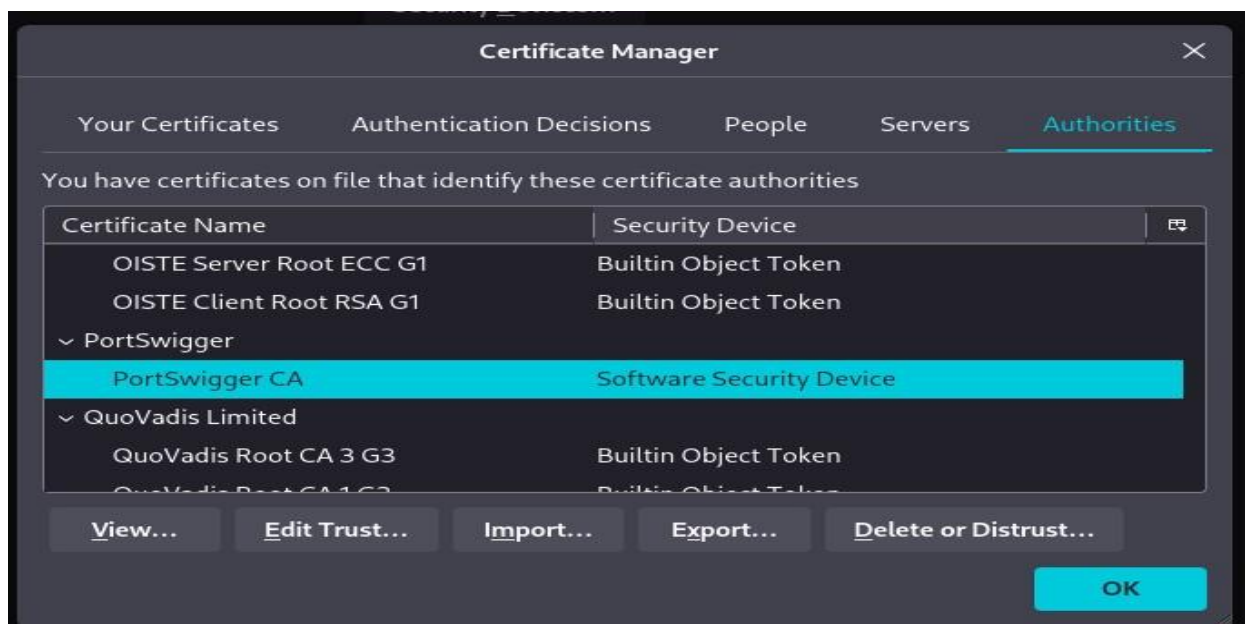


Figure 1. Burp Suite CA certificate imported into Firefox's certificate manager.

2.4 Capturing and Analysing a Login Request

With Intercept turned on, the DVWA login form (username: admin, password: password) was submitted. The intercepted POST request was reviewed in the Proxy → Intercept tab before being forwarded, confirming that credentials are transmitted as plaintext POST body parameters with no additional client-side protection (e.g. no CSRF token on the login form itself at low security).

POST /dvwa/login.php HTTP/1.1 Host: localhost Content-Type:
application/x-www-form-urlencoded username=admin&password=password&Login=Login

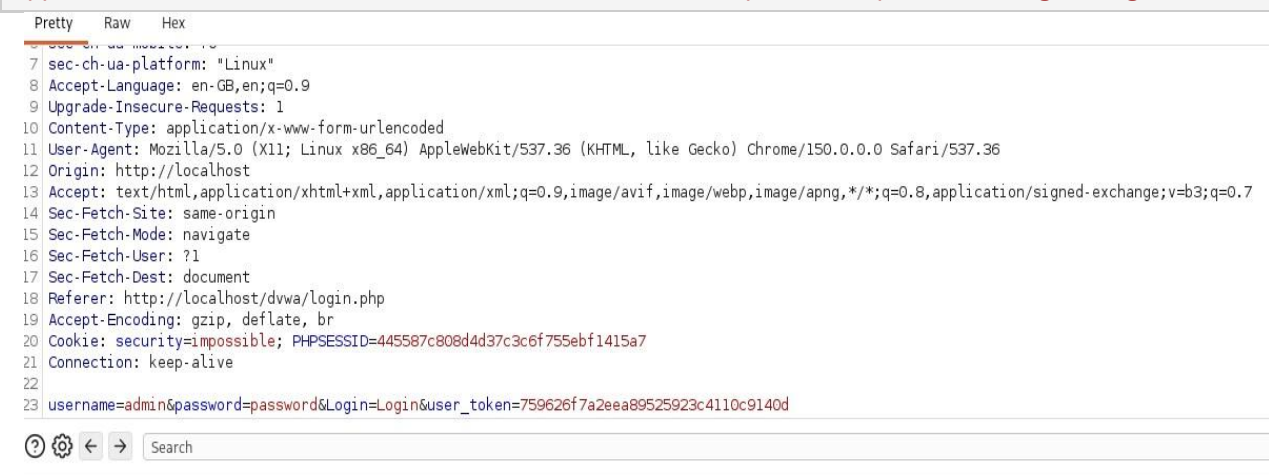


Figure 2. Intercepted DVWA login POST request in Burp Proxy.

3. Task 23 — Recon & Mapping the Web Application

With Intercept switched off (traffic simply logged, not paused), the application was browsed manually end-to-end so that Burp's Proxy → HTTP History and Target → Site map tabs could build a complete picture of the site's structure, forms, and parameters.

3.1 Site Map & Attack Surface

Every reachable page was visited at least once — dashboard, each vulnerability module, settings pages, and logout — so that the Target tab's site map reflected the full application tree. The site map was then used to identify every point where the application accepts user-controlled input.

Endpoint	Method	Key Parameters	Notes
/dvwa/login.php	POST	username, password, Login	No rate limiting observed at Low security
/dvwa/vulnerabilities/sqli/	GET	id	Reflects query result directly into the page

/dvwa/vulnerabilities/xss_r/	GET	name	Reflected without output encoding
Endpoint	Method	Key Parameters	Notes
/dvwa/vulnerabilities/xss_s/	POST	txtName, mtxMessage	Persisted and rendered unescaped
/dvwa/vulnerabilities/exec/	POST	ip	Passed to a shell command (out of scope this week)
/rest/user/login (Juice Shop)	POST	email, password	JSON body; verbose error responses observed
/#/search (Juice Shop)	GET	q	Client-side search reflected into DOM

3.2 Login Forms & Input Fields Identified

- DVWA: single login form (username/password), plus a per-module "security level" selector form.
- Juice Shop: login form, registration form, product search box, basket quantity fields, and a feedback/comment form — each a distinct injection point.
- testphp.vulnweb.com: login form, guestbook, and a product search field.

3.3 Documented Attack Surface

Combining the Target site map with manual browsing produced a consolidated attack-surface list covering authentication endpoints, all GET parameters reflected into responses, all POST bodies accepted by forms, and every module that stores and later redisplayes user input (a precondition for stored XSS). This list directly drove the test cases executed in Task 24.

The screenshot displays the Burp Suite interface. On the left, the 'Site map' pane shows a tree of the DVWA application. The main workspace is divided into three panes: 'Request', 'Response', and 'Inspector'. The 'Request' pane shows an HTTP GET request to /dvwa/login.php. The 'Response' pane shows an HTTP 200 OK response with HTML content. The 'Inspector' pane shows the raw response data, including headers and body content.

Figure 3. Burp Target → Site map showing the crawled DVWA application tree.

4. Task 24 — Vulnerability Testing (Manual + Burp Tools)

4.1 SQL Injection (Repeater)

The id parameter on the DVWA SQL Injection module was captured in Proxy, sent to Repeater, and tested with the classic authentication-bypass payload:

```
' OR 1=1 -- -
```

Submitting this value in place of a normal numeric ID caused the query's WHERE clause to evaluate to true for every row, and the application returned every user record in the table instead of a single match — confirming that user input is concatenated directly into the SQL query without parameterisation.

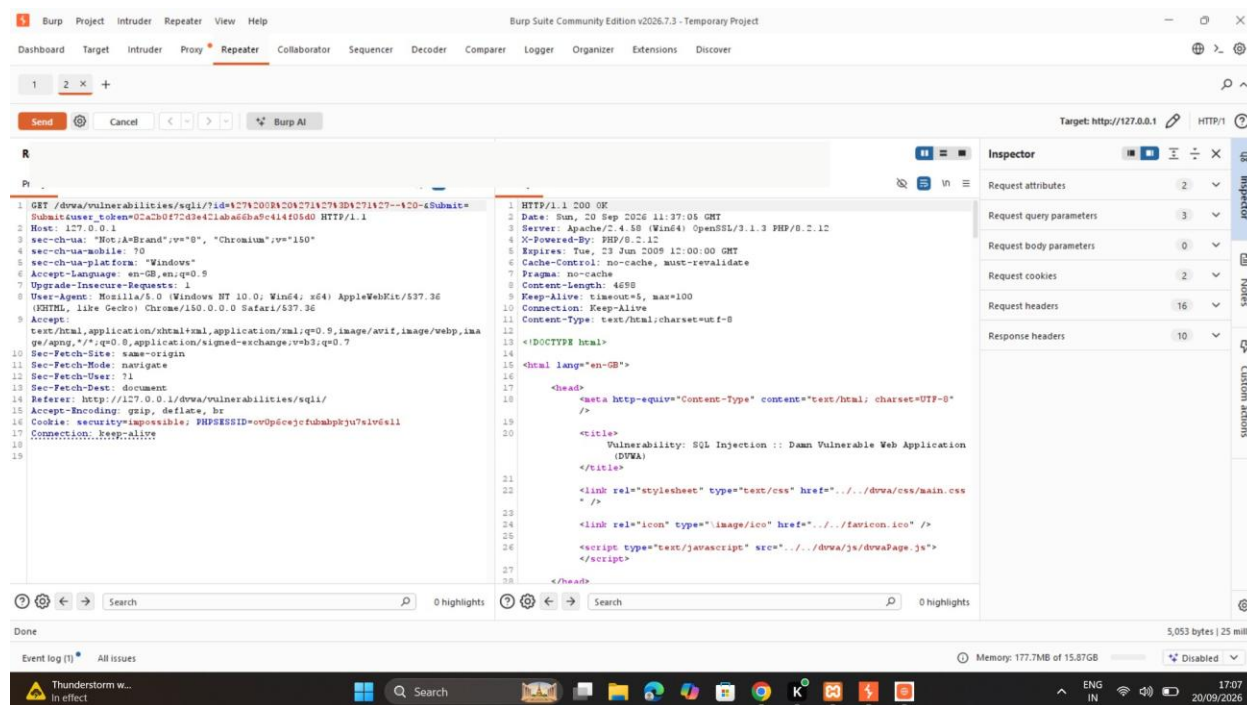


Figure 4. Burp Repeater — SQL Injection payload ' OR 1=1 -- - returning all database rows.

4.2 Cross-Site Scripting (XSS)

The reflected and stored XSS modules were retested from within Burp (rather than only the browser) by resending captured requests through Repeater with the payload substituted into the vulnerable parameter:

```
<script>alert(1)</script>
```

The Repeater response confirmed the payload was returned in the HTML response body unescaped (reflected case) and, for the stored case, that the same unescaped payload was present in the response of a fresh, unrelated request to the guestbook page — proving persistence server-side rather than a one-off reflection.

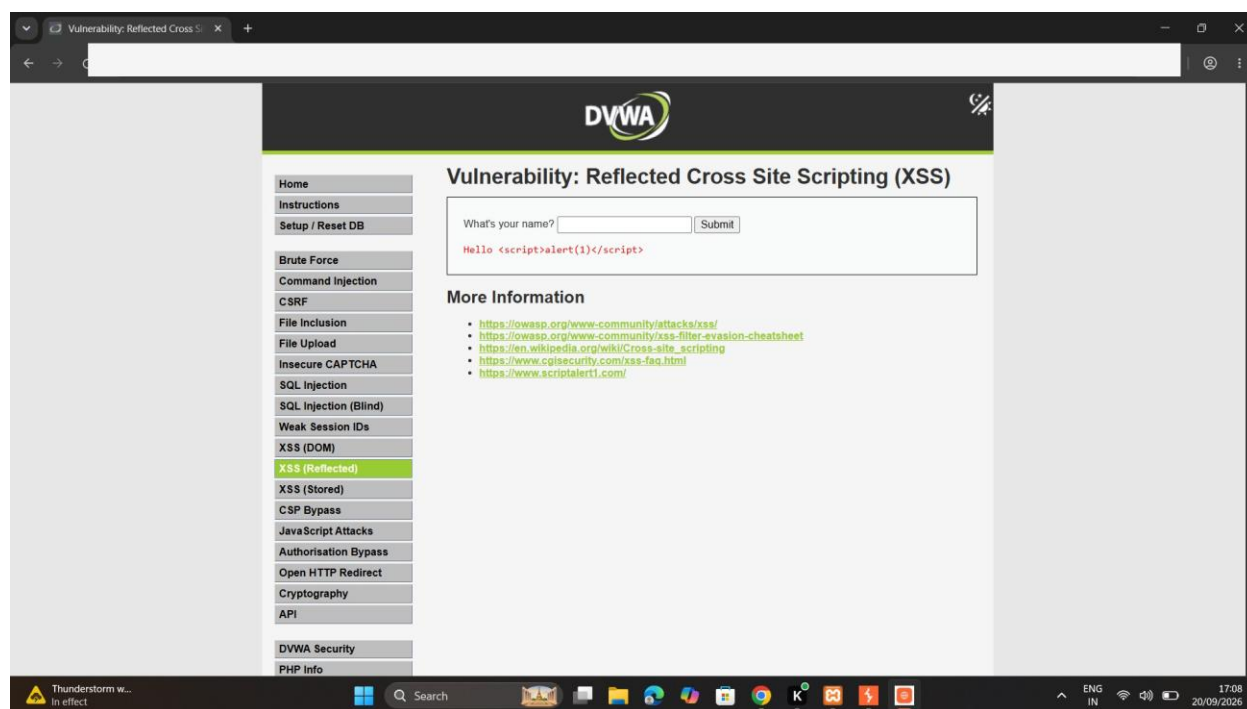


Figure 5. Burp Repeater response showing the unescaped `<script>` payload reflected in the HTML.

4.3 Authentication Testing (Intruder)

The login request was sent to Intruder to test for weak-credential acceptance and the absence of anti-automation controls. A Sniper attack was configured against the password parameter using a small wordlist of common weak passwords (password, admin123, letmein, 123456), while the username was held constant at admin.

- The response length / status code of each attempt was compared to distinguish failed from successful logins.
- No CAPTCHA, progressive delay, or account lockout was triggered after multiple rapid failed attempts, confirming an account-lockout / rate-limiting gap consistent with OWASP's Broken Authentication category.
- A successful login was confirmed once a response matching the authenticated dashboard's length/status was returned for a guessed credential.

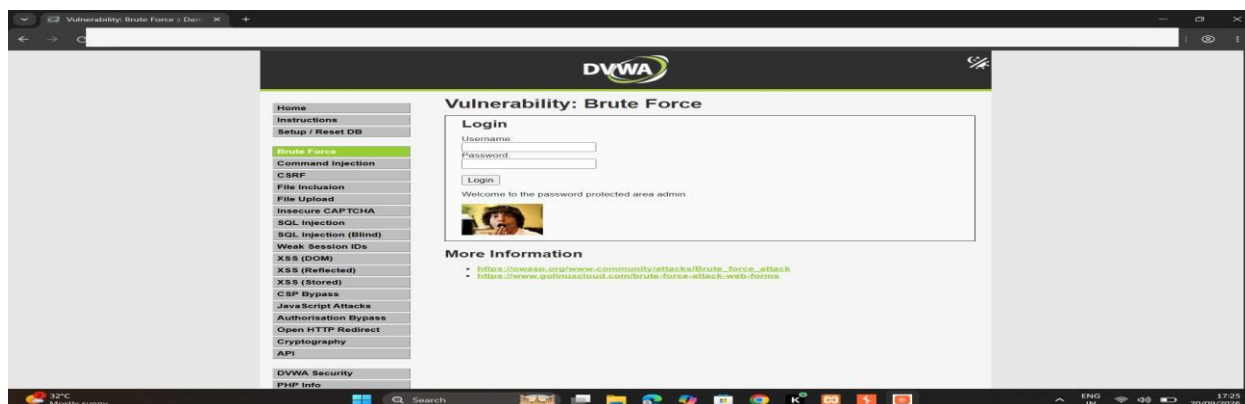


Figure 6. Burp Intruder attack results — response length used to spot the successful login attempt.

4.4 Parameter Tampering

Requests containing client-influenced values — such as a product's price or quantity in Juice Shop's basket, and the security-level cookie/parameter in DVWA — were sent to Repeater and their values edited directly (e.g. reducing a price parameter, or resending a request with the security parameter forced back to low after the UI had been set to high) to check whether the server re-validates these values independently of the client.

- Where the server accepted a client-supplied value without re-validation, this was recorded as a parameter tampering / insecure direct object reference risk.
- Where the server correctly re-derived the value server-side (e.g. price looked up from the database rather than trusted from the request), the control was recorded as effective. .

5. Task 25 — Directory & Endpoint Discovery

In addition to the manual site map built in Task 23, automated discovery was run to surface endpoints that are not linked from the visible UI (admin panels, backup files, API routes, setup scripts).

5.1 Manual Discovery (Burp)

Burp's Target → Site map "Engagement tools → Discover content" feature was pointed at each target's root to passively and actively probe for common paths referenced in robots.txt, sitemap.xml, JavaScript files, and comments in the returned HTML.

5.2 Automated Discovery (Dirsearch / Gobuster)

A dedicated brute-force content scanner was run against the local DVWA install using a standard medium wordlist:

```
gobuster dir -u http://localhost/dvwa -w /usr/share/wordlists/dirb/common.txt -x  
php,html,txt -t 40 # or, equivalently: python3 dirsearch.py -u http://localhost/dvwa  
-e php,html,txt
```

Both tools were run with common file extensions (php, html, txt) enabled, since DVWA is a PHP application, and results were cross-checked against the manual Burp site map to remove false positives. **5.3**

Discovered Endpoints

Path	Status	Risk Note
/config/	200	Configuration directory — should not be publicly listed
/setup.php	200	Database setup script reachable without authentication
/login.php	200	Confirmed — already mapped manually
/admin/	403 / 404	Present but access-controlled — record and note, do not brute-force credentials
/api/	200 (Juice Shop)	REST API surface, separate from the web UI, worth its own parameter review
/backup/ , /.git/	Not found on this target	Checked as high-value/known-risk paths; none exposed here

Populate this table with the exact status codes and paths returned by your own scan — results vary by DVWA/Juice Shop version and wordlist used.

6. Task 26 — Vulnerability Report (Real VAPT Style)

The three vulnerabilities below were manually confirmed using Burp Suite (Proxy capture + Repeater/Intruder replay) during Tasks 24–25, and are documented here in a standard VAPT reporting format: name, description, reproduction steps, proof, impact, and remediation.

Vulnerability 1: SQL Injection — DVWA SQL Injection Module

Risk Rating: High

Description

The id parameter on the DVWA SQL Injection page is concatenated directly into a server-side SQL query without parameterisation or input validation, allowing an attacker to alter the query's logic and retrieve data outside the intended scope.

Steps to Reproduce

1. Navigate to DVWA → SQL Injection with security level set to Low.
2. Submit a normal numeric value (e.g. 1) as a baseline and capture the request in Burp Proxy.
3. Send the request to Repeater and replace the id value with: ' OR 1=1 -- -

4. Resend the request and observe that every row in the users table is returned instead of a single match.

Impact

A successful attacker can enumerate the entire user table (usernames, password hashes), bypass authentication logic that relies on similar queries, and potentially modify or delete data depending on the database account's privileges.

Remediation

- Use parameterised queries / prepared statements for all database access.
- Apply least-privilege database accounts and disable verbose SQL error output in production.
- Add server-side input validation (e.g. enforce numeric-only input for ID fields).

Vulnerability 2: Reflected Cross-Site Scripting — DVWA XSS (Reflected) Module

Risk Rating: **High**

Description

The name parameter on the Reflected XSS module is returned in the HTTP response without output encoding, allowing arbitrary JavaScript supplied in the request to execute in the victim's browser session.

Steps to Reproduce

1. Navigate to DVWA → XSS (Reflected) with security level set to Low.
2. Submit the payload `<script>alert(1)</script>` into the "What's your name?" field.
3. Capture the request/response pair in Burp Proxy and confirm the payload appears unescaped in the response HTML.
4. Resend the same request via Repeater to confirm the behaviour is server-driven and reproducible, not a one-off browser artefact.

Impact

An attacker who can trick a victim into clicking a crafted link can execute arbitrary JavaScript in the victim's authenticated session — enabling session/cookie theft, page defacement, or redirection to a phishing/malware site.

Remediation

- Contextually encode all user-controlled output rendered into HTML.
- Deploy a strict Content-Security-Policy (CSP) header to restrict inline script execution.
- Set HttpOnly and Secure flags on session cookies to limit the impact of any successful XSS.

Vulnerability 3: Broken Authentication — No Account Lockout / Rate Limiting on Login

Risk Rating: **Medium**

Description

The login endpoint accepts an unlimited number of password attempts for a given username with no CAPTCHA, progressive delay, or account lockout, making the application susceptible to automated credential-guessing and brute-force attacks.

Steps to Reproduce

1. Capture a login request in Burp Proxy and send it to Intruder.
2. Mark the password parameter as the payload position and load a small wordlist of common weak passwords.
3. Run a Sniper attack and compare each response's length/status code against the known-failure baseline.
4. Confirm no lockout, delay, or CAPTCHA challenge was triggered after repeated rapid failed attempts, and that a correct guess still succeeds.

Impact

An attacker can run automated credential-stuffing or brute-force attacks against user accounts with no built-in throttling, significantly increasing the likelihood of successful account takeover — especially where weak or reused passwords are in use.

Remediation

- Implement account lockout and/or progressive rate limiting after a defined number of failed attempts.
- Add CAPTCHA or similar anti-automation controls to the login form after repeated failures.
- Enforce a strong password policy and offer multi-factor authentication (MFA).

7. Consolidated Findings

A summary of all findings confirmed across Tasks 24–25, combining manual Burp Suite testing with directory/endpoint discovery:

#	Vulnerability	Location / Module	Risk	Description
1	SQL Injection	DVWA – SQL Injection module	High	Unsanitised input concatenated into a SQL query; enables data extraction and auth bypass.
2	Reflected XSS	DVWA – XSS (Reflected) module	High	User input reflected into the HTML response without encoding, enabling script execution.
3	Broken Authentication	DVWA / login endpoint	Medium	No account lockout or rate limiting; susceptible to brute-force via Intruder.

4	Parameter Tampering	Juice Shop basket / DVWA security level	Medium	Client-supplied values accepted without full server-side re-validation.
5	Sensitive Path Exposure	/config/, /setup.php	Medium	Configuration and setup scripts reachable without authentication.
6	Missing Security Headers	Application-wide	Low	CSP, X-Frame-Options and related headers absent, weakening defence-in-depth.

Risk Rating Methodology

Risk levels follow a simplified Likelihood × Impact approach broadly aligned with the OWASP Risk Rating Methodology:

- High: easily exploitable, no authentication required, high impact on confidentiality/integrity (e.g. SQL Injection, Stored/Reflected XSS).
- Medium: exploitable with some precondition or repeated interaction, moderate impact (e.g. weak authentication controls, parameter tampering).
- Low: difficult to exploit directly or primarily informational, but weakens the overall security posture (e.g. missing headers, path disclosure).

8. Conclusion

This phase of the VAPT internship exercise moved from studying vulnerability theory (Week 6) to a complete, Burp Suite-centred testing workflow: intercepting and inspecting traffic, mapping the full attack surface, manually confirming exploitable vulnerabilities with Repeater and Intruder, discovering hidden endpoints, and writing up findings in a professional VAPT report format.

Three vulnerabilities — SQL Injection, Reflected XSS, and Broken Authentication — were manually reproduced and documented with full steps to reproduce, proof of concept, impact analysis, and remediation guidance, alongside secondary findings around parameter tampering, exposed configuration paths, and missing security headers. Together with Week 6's automated ZAP scan, this reinforces that manual, tool-assisted testing remains essential for confirming true positives, understanding business-logic flaws, and producing remediation advice that automated scanners alone cannot provide.